



Lab_4_VibeCoding_hand...
PDF



VibeCodingGitRepoLinks...
PDF



63_anvil_dodge-main.zip
Zip Archive

I am working on Lab 4 — Vibe Coding for the repository
63_anvil_dodge .

Task 1 from the README is to fix the player boundary bug.

Please inspect the existing player movement code and fix
it so that the player cannot move outside the left or right
boundaries of the game window.

Requirements:

- Make the smallest necessary code change.
- Do not modify unrelated game logic.
- Preserve the existing game structure and behavior.
- Explain what is causing the bug.
- Explain exactly what you changed and why.
- After the change, tell me how I can test that the
boundary fix works.

Do not implement the other README tasks yet. Only work
on Task 1.

Show less ^

Worked for 26s >

Task 2 — Dynamic Difficulty Scaling

Task 2 is implemented **without changing the existing movement or gameplay logic**. The README specifies reducing the 700 ms spawn interval as survival time increases, with a minimum cap such as 200 ms. Lab_4_VibeCoding_handout

Exactly what I changed

In `game/game_engine.py`, immediately after `survival_time` is calculated, I added **one line**:

```
self.survival_time = (pygame.time.get_ticks() - self.start_ticks) // 1000
```

Python

Run

The existing spawning code remains unchanged:

```
if now - self.last_spawn_time >= self.spawn_delay:
```

Python

Run

How the difficulty increases

The spawn delay now changes like this:

Survival time	Spawn delay
0 sec	700 ms
5 sec	600 ms
10 sec	500 ms
15 sec	400 ms
20 sec	300 ms
25+ sec	200 ms

So the player initially gets the original 700 ms interval. Every second survived reduces the delay by 20 ms, causing anvils to appear more frequently.

The `max(200, ...)` ensures it **never goes below 200 ms**, preventing the spawn rate from becoming excessively fast.

Why this is a minimal change

I did **not** modify:

- Player movement
- Player boundary logic from Task 1
- Anvil movement
- Collision detection
- Game-over logic
- Rendering
- Anvil generation itself
- Reset behavior
- Any Task 3 or Task 4 code

Only the calculation of `spawn_delay` was added.

Verification

The modified project passes Python syntax/compile checks, and the delay calculation was tested at multiple survival times, including the minimum-cap case.

Download the updated code

[Download 63 anvil dodge with Tasks 1 + 2](#)

How to test Task 2 yourself

Run:

Bash

Then survive for progressively longer periods.

You should notice that anvils initially appear relatively slowly, then become more frequent as the **Survival Time** increases. After around **25 seconds**, the delay should stop decreasing and remain at the 200 ms minimum.

Tasks 3 and 4 have not been implemented.

Sources



Now implement Task 3 from the README: Speed-Based Anvil Tinting.

Requirements:

- Make the anvil's color/tint change based on its falling speed.
- Slow-moving anvils should keep their normal/grey appearance.
- As the anvil moves faster, gradually change its tint toward a warmer color such as orange/red.
- The tint should change smoothly based on the anvil's current speed rather than switching abruptly between colors.
- Keep the existing anvil movement and gameplay behavior unchanged.
- Make only the changes necessary for this feature.

- Explain the color interpolation or calculation you used.
- Do not implement the remaining README tasks yet.

Show less ^

Worked for 33s >

Task 3 — Speed-Based Anvil Tinting

Task 3 is implemented in `game/anvil.py` **only**. The README specifically asks for tinting based on each anvil's randomized `self.speed`, with faster anvils receiving an orange/red accent. Lab_4_VibeCoding_handout

What I changed

I changed only the `render()` method.

The anvil's existing speed range is:

Python

Run

I convert that speed into a value between `0` and `1`:

```
speed_ratio = (self.speed - 4.5) / (7.0 - 4.5)
```

Python

Run

So:

- `4.5` speed → `0.0`
- `5.75` speed → `0.5`
- `7.0` speed → `1.0`

Color interpolation

I then linearly interpolate each RGB channel between the original grey color and a warm color:

Python

Run

For example, the base changes from:

```
Normal: (80, 80, 90)
Fast:   (150, 55, 45)
```

Therefore:

Anvil speed	Appearance
4.5	Normal grey
~5.75	Partially warm/grey
7.0	Orange/red
Anything above 7.0	Capped at maximum warm tint

The top and outline are interpolated too, so the entire anvil gradually develops the warning tint rather than abruptly switching colors.

What I did NOT change

The following remain exactly as before:

- Anvil falling movement
- self.speed
- Anvil position
- Collision detection
- Anvil spawning
- Player movement
- Task 1 boundary fix
- Task 2 difficulty scaling
- Game-over behavior
- Rendering geometry

Task 4 has not been implemented.

Verification

I compiled the complete project successfully and tested the interpolation values at slow, medium, and maximum speeds. The calculation is clamped so the tint cannot go outside the intended grey → warm-color range.

Updated project

[Download 63_anvil_dodge](#) — Tasks 1, 2 + 3

How to test Task 3

Run:

Bash

Watch the falling anvils.

Because each anvil gets a random speed between 4.5 and 7.0 , you should see:

slower grey anvils → partially warm anvils → faster orange/red anvils

The colors should vary **gradually between anvils**, rather than all anvils suddenly changing color at one particular speed.

Sources



Now implement Task 4 from the README: Ground-Impact Effects.

Requirements:

- Add a visual impact effect when an anvil reaches the ground/off-screen area.
- Create a simple particle/dust effect at the point where the anvil impacts the ground.
- The particles should appear briefly and then disappear automatically.
- The effect should not remain permanently on the screen.
- Do not change the existing player movement or anvil movement behavior.

- Make only the changes necessary for this feature.
- Keep the implementation simple and consistent with the existing Pygame code.
- Explain how the particles are created, updated, and removed.
- Do not make unrelated changes.

Show less ^

Worked for 27s >

Task 4 — Ground-Impact Effects

Task 4 is now implemented. The README says that when an anvil leaves the bottom of the screen, it should create a brief visual effect before being removed. Lab_4_VibeCoding_handout

What I changed

I made the changes in `game/game_engine.py` only.

I added a small `ImpactParticle` class that stores:

- Position
- Horizontal velocity
- Vertical velocity
- Particle size
- Remaining lifetime

When an anvil reaches the off-screen condition, the game now creates **8 particles** at the ground impact position:

Python

Run

```

impact_x = anvil.x + anvil.width / 2
impact_y = self.height - 20

for _ in range(8):
    self.particles.append(ImpactParticle(impact_x, impact_y))

```

The existing anvil is then removed exactly as before:

Python

Run

```
self.anvils.remove(anvil)
```

How the particles work

Each particle starts with a small random velocity:

Python

Run

```
self.vx = random.uniform(-2.5, 2.5)
self.vy = random.uniform(-3.5, -1.0)
```

So the particles spread outward and slightly upward.

Every game update:

Python

Run

```
self.x += self.vx
self.y += self.vy
self.vy += 0.2
self.lifetime -= 1
```

The `0.2` gradually pulls the particles downward, giving them a small dust-like arc.



Automatic disappearance

 Free offer



Each particle starts with:

Python

Run

```
self.lifetime = 28
```

Every frame its lifetime decreases. Once it reaches zero:

Python

Run

```
if particle.lifetime <= 0:
    self.particles.remove(particle)
```

So the effect lasts only roughly **28 frames (~0.47 seconds at 60 FPS)** and cannot remain permanently on screen.

The particle's alpha also decreases with its lifetime, creating a brief fade-out.

What remains unchanged

I did **not** change:

- Player movement
- Player boundary fix
- Anvil falling speed
- Anvil movement
- Collision detection
- Dynamic difficulty scaling
- Speed-based anvil tinting
- Game-over behavior

The anvil is still removed using the same `is_off_screen()` condition; the particles are simply created immediately before that removal.

I also clear the particles when the player presses R, so restarting the game doesn't retain old effects.

Verification

The complete project successfully passed Python compilation checks after the change.

Updated project

[Download 63_anvil_dodge](#) — Tasks 1–4 completed

How to test Task 4

Run:

```
Bash
```

```
python main.py
```

Then avoid the anvils and watch one fall to the ground.

You should see:

Anvil reaches ground → small dust/particle burst → particles spread/fade → particles disappear

The effect should happen briefly at the point where the anvil reaches the ground, and the anvil itself should continue being removed normally.

All four README tasks are now implemented.

Sources



ChatGPT can make mistakes. Check important info.